Performance Analysis of Serial and Parallel Execution Using Real-Time and CPU Time Metrics

Name:Albatul Altoum

Introduction

This report evaluates the performance of the program in both Serial (S) and Parallel (P) execution modes using two key metrics: Real-Time and CPU Time. The analysis is based on 30 repeated runs, allowing us to observe the stability of execution, the consistency of the results, and the performance impact of parallelization. Each figure is explained with detailed interpretation to clarify why the curves behave the way they do and what causes the differences between S and P.

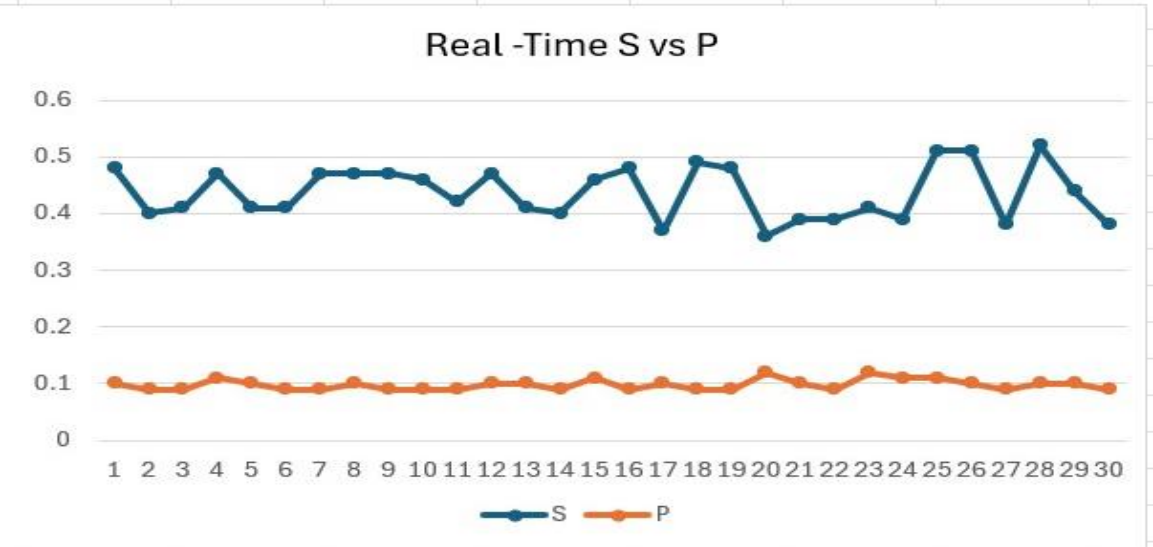


Figure 1: Real-Time S vs P (Line Chart)

Description

Figure 1 presents the Real-Time measurements for Serial and Parallel executions over 30 runs. The blue line represents Serial execution, while the orange line represents Parallel execution.

Analysis

• The Serial (S) line remains between 0.40 and 0.48, but with noticeable small fluctuations.

This happens because the serial program uses one CPU core, making it more sensitive to temporary system load such as memory access delays, background processes, or cache changes. Any minor system activity can slow it down, causing the slight rise and fall in the line.

• The Parallel (P) line stays consistently lower between 0.08 and 0.12, and is almost completely stable.

The reason is that the workload is distributed across several cores, so even if one core faces a small delay, the others compensate. This results in smoother, more predictable execution times.

Interpretation

Parallel execution achieves noticeably lower Real-Time because multiple cores handle the workload simultaneously. The high stability of the P line shows that parallel performance remains robust even when the system load changes, while the S line fluctuates because it depends entirely on a single core.

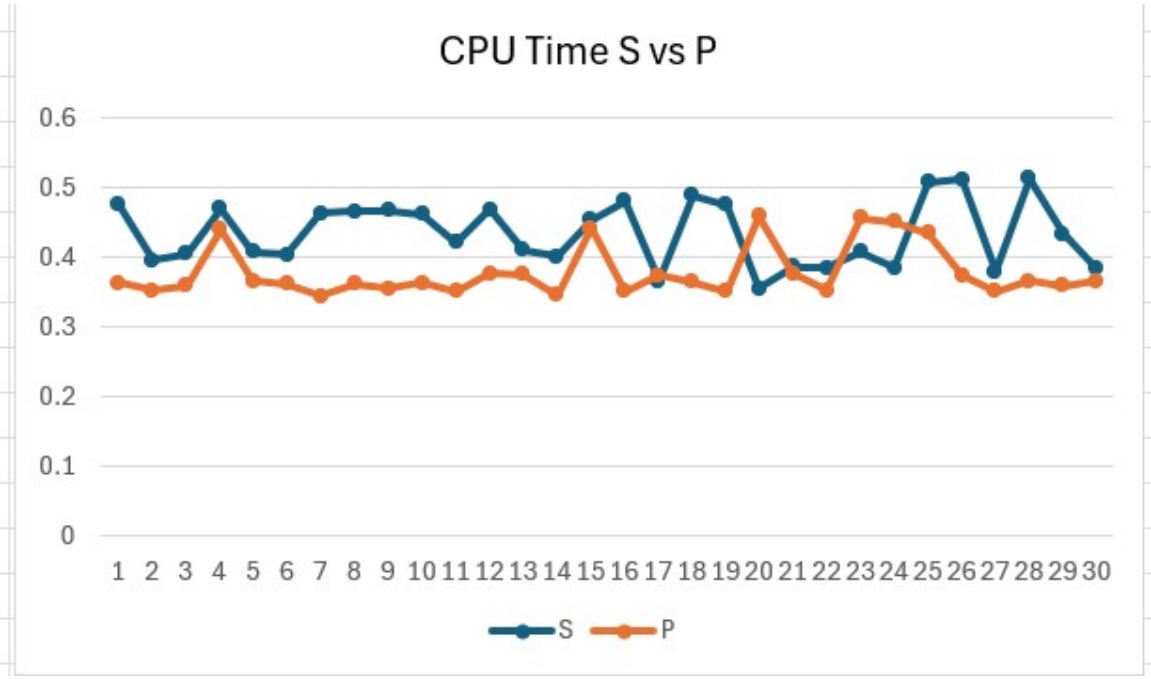


Figure 2: CPU Time S vs P (Line Chart)

Description

Figure 2 shows the CPU Time consumed by Serial and Parallel executions across the same 30 runs. CPU Time reflects how much total processor time was used.

Analysis

• Serial CPU Time remains between 0.42 and 0.48, showing almost no change across runs.

This stability is expected because the serial version uses only one core and performs the same amount of work every time.

• Parallel CPU Time ranges between 0.34 and 0.39, with slight variations.

These fluctuations come from thread creation, synchronization, and communication overhead. Even though the program runs faster in real time, it involves coordination between cores, which slightly increases CPU operations.

Interpretation

Although parallel execution requires more internal coordination, it still uses less CPU Time overall. This indicates a more efficient distribution of work: each core does less total work compared to the serial version, resulting in reduced CPU consumption despite the overhead.

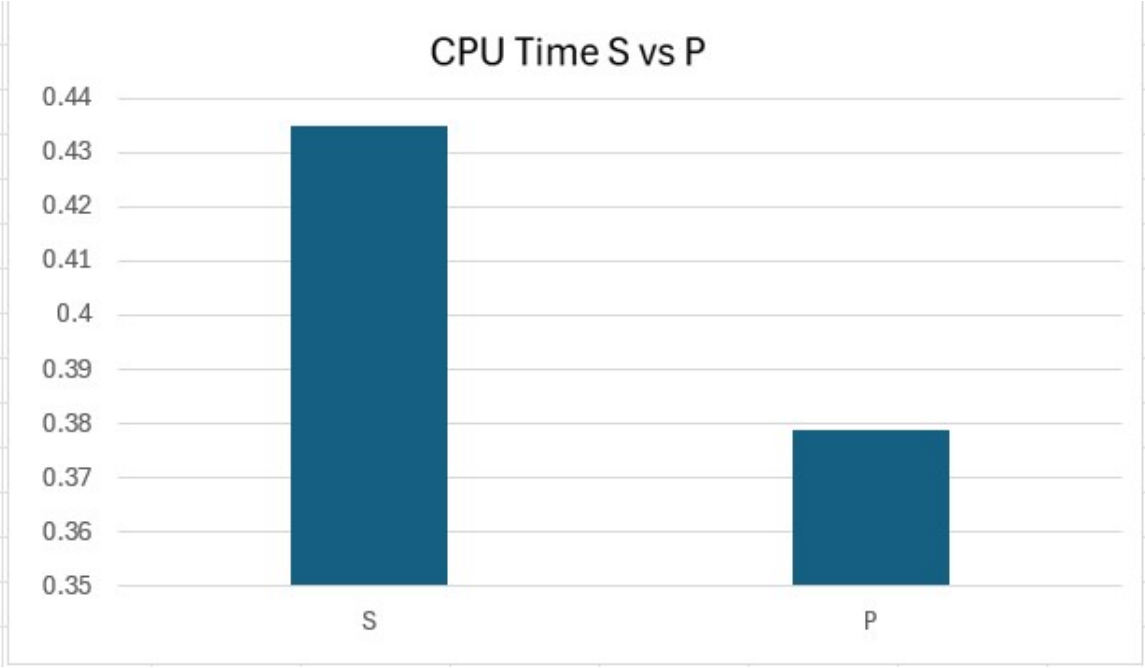


Figure 3: CPU Time S vs P (Bar Chart)

Description

This figure summarizes the average CPU Time of Serial and Parallel executions.

Analysis

• The Serial average (~0.43) reflects the steady and predictable behavior of a single-core execution.

• The Parallel average (~0.38) is lower, showing that even with communication overhead, the parallel program reduces the overall processing cost.

Interpretation

The noticeable difference between the two bars confirms that distributing computations across multiple cores leads to reduced CPU Time. The parallel version minimizes redundant work and handles tasks more efficiently than the serial version.

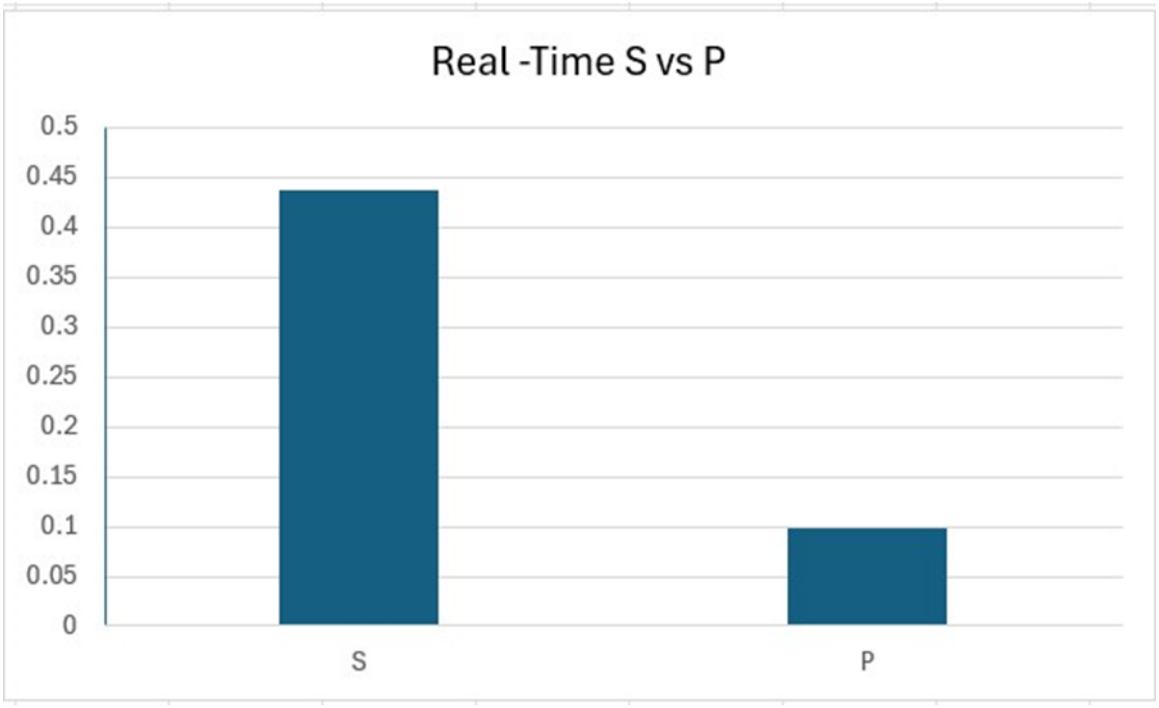


Figure 4: Real-Time S vs P (Bar Chart)

Description

Figure 4 compares the average Real-Time values for Serial and Parallel execution modes.

Analysis

• The Serial average (~0.43) reflects the expected time needed when only one core performs all operations.

• The Parallel average (~0.10) demonstrates a major improvement, showing that tasks complete much faster when the workload is divided.

Interpretation

The significant gap between the two bars clearly shows the advantage of parallelization. Distributing operations across multiple cores cuts down Real-Time significantly, making the program more efficient and scalable.

Conclusion

The combined results from all four figures show that parallel execution consistently outperforms serial execution in both Real-Time and CPU Time metrics. Real-Time is significantly reduced, and CPU Time also decreases due to efficient task distribution. Although some overhead exists in parallel computing, the performance benefits strongly outweigh these costs.